Cargo Quote Automation

Full workflow, step by step (1a – 9) — with worked examples

This document walks through the complete automation pipeline for turning a client's shipment request — submitted through Translog's own client-facing web portal or by email — into a confirmed booking, using an LLM (Qwen Plus 0728 via OpenRouter) for email understanding and a reverse-engineered internal WebCargo (Freightos) API for rate search and booking.

The big picture, in three jobs: (1) get the request — from the client, however they send it — (2) find the best rate — search WebCargo, filter, rank — (3) confirm and book — client says yes, the system locks it in. Everything below is what happens inside those three jobs.

One identity to keep straight throughout: WebCargo has exactly one user — Translog itself. Every client (Mehul, or anyone else) is Translog's user, not WebCargo's. No client identity or credential ever reaches WebCargo — only Translog's own stored login does, reused across every client's request.

Workflow at a glance

Step	Name	One-line job
1a	Client web portal	Client fills a structured shipment form directly.
1b	Client email	Client emails a free-text request instead.
—	Qwen Plus 0728 extraction	Sub-step of 1b: turns email text into structured fields.
2	Unified shipment record	Both paths merge into one canonical JSON shape.
3	Validate & fill gaps	Checks required fields; loops back to the client if incomplete.
4	Authenticate WebCargo session	Ensures a valid login/token exists for Translog's account.
5	Call internal search API	Sends the query, gets back every rate option.
6	Filter, rank, select	Removes ineligible options, scores the rest, picks one.
7	Send accept / reject prompt	Shows the client one recommendation, waits for a decision.
8a	Accept → book	Confirms the booking via the internal API.
8b	Reject → next best	Loops back to step 6, excluding the rejected option.

Two of these arrows are feedback loops, not dead ends: step 3 can loop back to the client for more info, and step 8b can loop back to step 6 for the next-best option. Both exist because the real email thread this project is modeled on needed exactly these two kinds of back-and-forth.

1a Client web portal

What it does: Client fills a form instead of writing a paragraph. Every field the business needs is captured up front: origin, destination, weight, dimensions, commodity, chemical yes/no, MSDS upload, piece count, delivery type.

Important distinction: this is a brand-new page Translog would build — not the WebCargo/Freightos website from the screen recording. Clients never see WebCargo; they only ever see this form and, later, a yes/no prompt.

Example — Mehul fills the form:

```
origin: Ahmedabad
destination: Bahrain (Hidd Industrial Area)
weight: 500 kg
dimensions: 24 x 34 x 6 inches
commodity: Polyisobutylene Additive
chemical: yes
MSDS: [uploaded]
pieces: 20 bags
delivery: door delivery
```

1b Client email

What it does: The other way a request can arrive — a normal email, for clients who don't use the portal. This step's only job is to notice a new email and hand off its raw text; it does not try to understand or extract anything itself.

Why it still exists: not every client will switch to the portal immediately. The system keeps watching the same inbox staff already use (via the Gmail API), so nothing changes for clients who keep emailing.

Example — the real first email in the Bahrain thread:

> *“Please provide a rate for 500 Kgs Cargo from Ahmedabad to Bahrain. Cargo dimension 24x34x6 inches. Cargo type Non Haz”*

Missing at this point: commodity, piece count, chemical status, delivery terms — all of which took several more emails to surface in the real thread.

• Qwen Plus 0728 extraction (sub-step of 1b)

What it does: Reads the raw email text and pulls out the same structured fields the portal form produces natively — filling a field only when the email actually states it, never guessing a plausible default.

Input (raw email text):

> *“Please provide a rate for 500 Kgs Cargo from Ahmedabad to Bahrain. Cargo dimension 24x34x6 inches. Cargo type Non Haz”*

Output (structured JSON):

```
{
  "origin": "Ahmedabad",
  "destination": "Bahrain",
  "weight_kg": 500,
  "dimensions_in": {"length": 34, "width": 24, "height": 6},
  "cargo_type": "Non Haz",
  "commodity": null,
  "pcs": null,
  "is_chemical": null,
  "delivery_type": null
}
```

A later reply in the same thread (“Commodity: POLYISOBUTYLENE ADDITIVE, No of Pcs: 20 Bags”) fills in two of the remaining blanks rather than starting a new record from scratch.

2 Unified shipment record

What it does: The point where the portal path and the email-plus-Qwen path stop being different things. Both get forced into one fixed JSON shape, so every step after this one only ever has to deal with a single format.

Shared schema:

```
{
  "request_id": "string",
  "source": "portal" | "email",
  "origin": "string",
  "destination": "string",
  "weight_kg": "number",
  "dimensions_in": {"length": "number", "width": "number", "height": "number"},
  "commodity": "string | null",
  "cargo_type": "string | null",
  "is_chemical": "boolean | null",
  "msds_attached": "boolean | null",
  "pcs": "number | null",
  "delivery_type": "door" | "airport" | null,
  "delivery_address": "string | null"
}
```

From the portal (complete) vs. from email + Qwen (gaps) — same shape either way:

Field	R-1001 (portal)	R-1002 (email)
commodity	Polyisobutylene Additive	null
is_chemical	true	null
pcs	20	null
delivery_type	door	null

Step 2 doesn't judge completeness — it only standardizes shape. Judging comes next.

---

3 Validate & fill gaps

What it does: Checks the unified record against a required-fields checklist. Complete → moves forward. Incomplete → loops back and asks the client, in one batched message, for everything still missing.

Checklist (some fields only required conditionally):

Field	Required when
origin, destination	Always
weight_kg, dimensions_in	Always
commodity	Always
cargo_type	Always

Field	Required when
is_chemical	Always
msds_attached	Only if is_chemical = true
pcs	Always
delivery_type	Always
delivery_address	Only if delivery_type = door

R-1001 (portal) passes straight through — every field, including the conditional MSDS one, is already present. R-1002 (email) fails — commodity, is_chemical, pcs and delivery_type are all missing, which triggers the loop back to the client:

> *“Thanks for your inquiry on the Ahmedabad→Bahrain shipment. To get you an accurate quote, could you confirm: (1) the commodity being shipped, (2) number of pieces, (3) whether this is a chemical product (and if so, please attach the MSDS), and (4) door delivery or airport pickup?”*

That one batched message replaces what took four separate emails over three days in the real thread.

4 Authenticate WebCargo session

What it does: Makes sure a valid login session exists for Translog's own WebCargo account before the next step tries to use it. Completely separate from validation — it never looks at any client's shipment data, only at Translog's stored credentials.

The identity model: WebCargo only ever knows about one account — Translog's. Every client is Translog's user, never WebCargo's. No client credential or identity is ever sent to WebCargo; the client's name/email is only used inside Translog's own system, to know who to reply to.

Sequence:

```
1. New validated record arrives (any client).
2. Check: is there a saved session token, still within its
   valid window?
3. If YES  -> reuse it, skip straight to step 5.
4. If NO   -> POST Translog's stored username/password to
              WebCargo's login endpoint -> save the returned
              cookie/token -> proceed to step 5.
```

Worked example across two clients:

Time	Trigger	What happens
9:00 AM	Mehul's Bahrain request	No token saved → log in → save token T1 (valid till 11:00 AM)
9:15 AM	A different client's KL request	T1 still valid → reuse it, no new login
11:05 AM	A third request	T1 expired → log in again → save token T2

Note: at no point does the client's identity enter this step. It manages exactly one token, for one account, shared across every client using the system.

---

5 Call internal search API

What it does: Uses the token from step 4 to send the shipment's search fields (origin, destination, weight, dimensions, date) to WebCargo's own backend endpoint — the one captured via browser DevTools, not a published API — and gets back every available rate option.

Request:

```
POST https://webcargonet.com/api/rates/search
Headers: { "Cookie": "session=T1" }
Body: {
  "origin": "AMD", "destination": "BAH",
  "weight_kg": 500,
  "dimensions_in": {"length": 34, "width": 24, "height": 6},
  "date": "2026-08-27"
}
```

Response (illustrative, shape based on the recorded results table):

```
{
  "results": [
    {"airline": "Etihad Airways", "product": "General Cargo",
     "total": 22779.90, "currency": "INR", "accepts_liquids": false},
    {"airline": "Emirates", "product": "GEN",
     "total": 20762.10, "currency": "INR", "accepts_liquids": true},
    {"airline": "Uzbekistan Airways", "product": "HY-250",
     "total": null, "currency": null, "accepts_liquids": true}
  ]
}
```


This step is a pure fetch — no filtering or decision-making happens here. A 401 or login redirect here means the token expired sooner than expected; control passes back to step 4 to refresh it and retry once.

6 Filter, rank, select

What it does: Two distinct passes over step 5's raw list.

Pass 1 — Filter (hard rules, no scoring):

Removes any option that genuinely cannot carry this cargo — e.g. the thread's own note that Etihad does not accept liquid products, or a row with missing/null rate data.

Applied to Mehul's shipment (chemical, but solid bags, not liquid): the liquid rule doesn't exclude Etihad here; Uzbekistan Airways is dropped for missing data. Survivors: Etihad, Emirates.

Pass 2 — Score and rank (judgment, on survivors only):

Airline	Total (INR)	Notes
Emirates	20,762.10	Lower total
Etihad	22,779.90	~2,000 more

No competing priority stated → lowest total wins → Emirates GEN selected.

If a preference is stated instead (e.g. the thread's later ask about a FlyDubai flight specifically), the ranking logic changes to filter for that carrier first, even if it isn't the cheapest — the whole shipment record is read here, not just weight and dimensions.

Output: one chosen option plus a short reason, e.g. “Emirates GEN — lowest total cost among carriers that accept this cargo type.”

---

7 Send accept / reject prompt

What it does: The one point where a human makes the actual decision. Formats step 6's single recommendation into a plain yes/no message and sends it back through whichever channel the client used originally.

Portal example:

```
Your quote is ready - Ahmedabad -> Bahrain, 500kg
Emirates, GEN service, via Dubai
Total: INR 20,762.10 (all-in, door delivery included)
Recommended because it's the lowest-cost option among
carriers accepting this cargo type.
[Accept]   [Reject]
```

Email example ends with: “Reply YES to confirm this booking, or NO if you'd like us to look at other options.”

Only ever one option is shown, never a list — showing several would just rebuild the manual process this project set out to remove. The pipeline pauses here for this shipment until a reply arrives; other shipments in flight are unaffected.

8a Accept → book via internal API

What it does: Takes the exact option shown in step 7 and calls WebCargo's internal booking endpoint with the same session token from step 4.

```
POST https://webcargonet.com/api/booking/confirm
Headers: { "Cookie": "session=T1" }
Body: {
  "rate_id": "EK-176-GEN-DXB",
  "shipment": { ...record from step 2/3... }
}
```

WebCargo returns a confirmation number (the same status the Quote Register in the recording shows moving to “Booked”). The client gets a final confirmation message, and this shipment's journey through the pipeline ends here.

8b Reject → next-best option

What it does: Loops back to step 6 rather than dead-ending — re-runs filter and rank with the rejected option excluded, folding in any reason the client gave.

Example: Mehul rejects Emirates and asks about the next available FlyDubai flight. The system excludes Emirates, re-filters/searches for FlyDubai specifically, produces a new single recommendation, and sends a fresh accept/reject prompt (back to step 7) for that option.

Worth deciding upfront: a cap on how many times this loop can repeat before the shipment is flagged for a human at Translog to take over manually.

The pattern behind every loop

Every loop in this workflow exists because a real message in the reference email thread showed the manual process looping too. The validation loop (step 3) replaces the clarifying emails Falguni had to send. The reject loop (step 8b) replaces Mehul's own follow-up asking about a different delivery option. The design principle is to automate exactly the back-and-forth that was already happening — not to invent new steps that weren't there before.

Identity summary to keep in mind while building

WebCargo has exactly one user: Translog. Clients are Translog's users, never WebCargo's. The client's name/email is used only inside Translog's own system, to route the final reply — it is never sent to, or checked by, WebCargo.